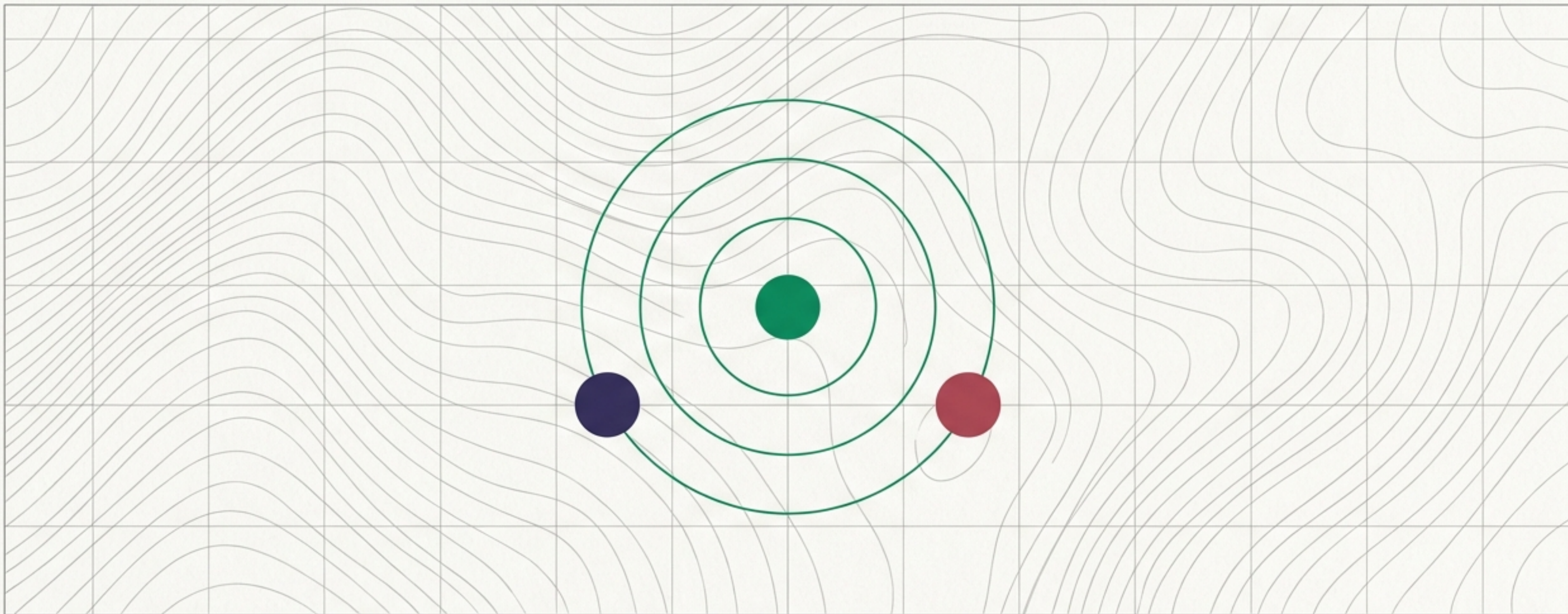




데이터의 소속을 묻다: KNN 알고리즘

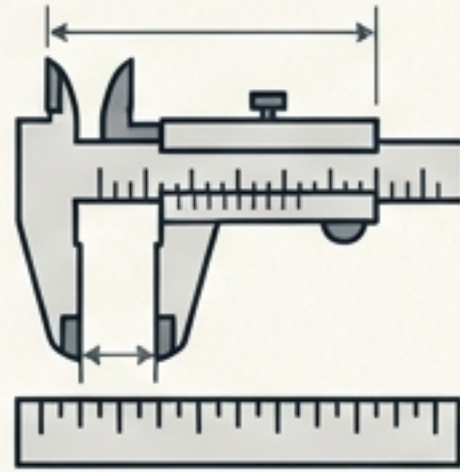
복잡한 수식 없이 '거리'와 '다수결'만으로 작동하는 가장 직관적인 인공지능 모델의 이해.

알고리즘의 이름에 숨겨진 직관적 의미



[K: 갯수]

+



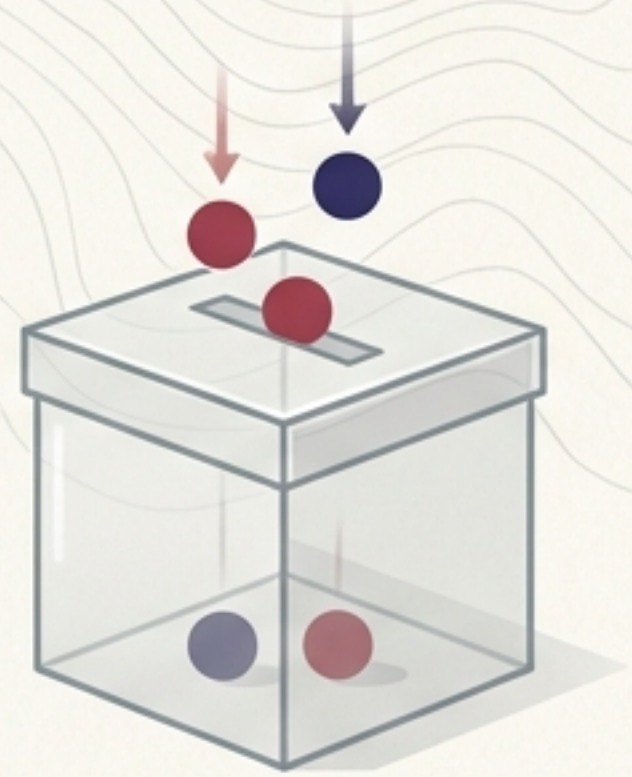
[Nearest: 최인접]

+



[Neighbor: 이웃]

=



= 다수결의 원칙
(Majority Rule)

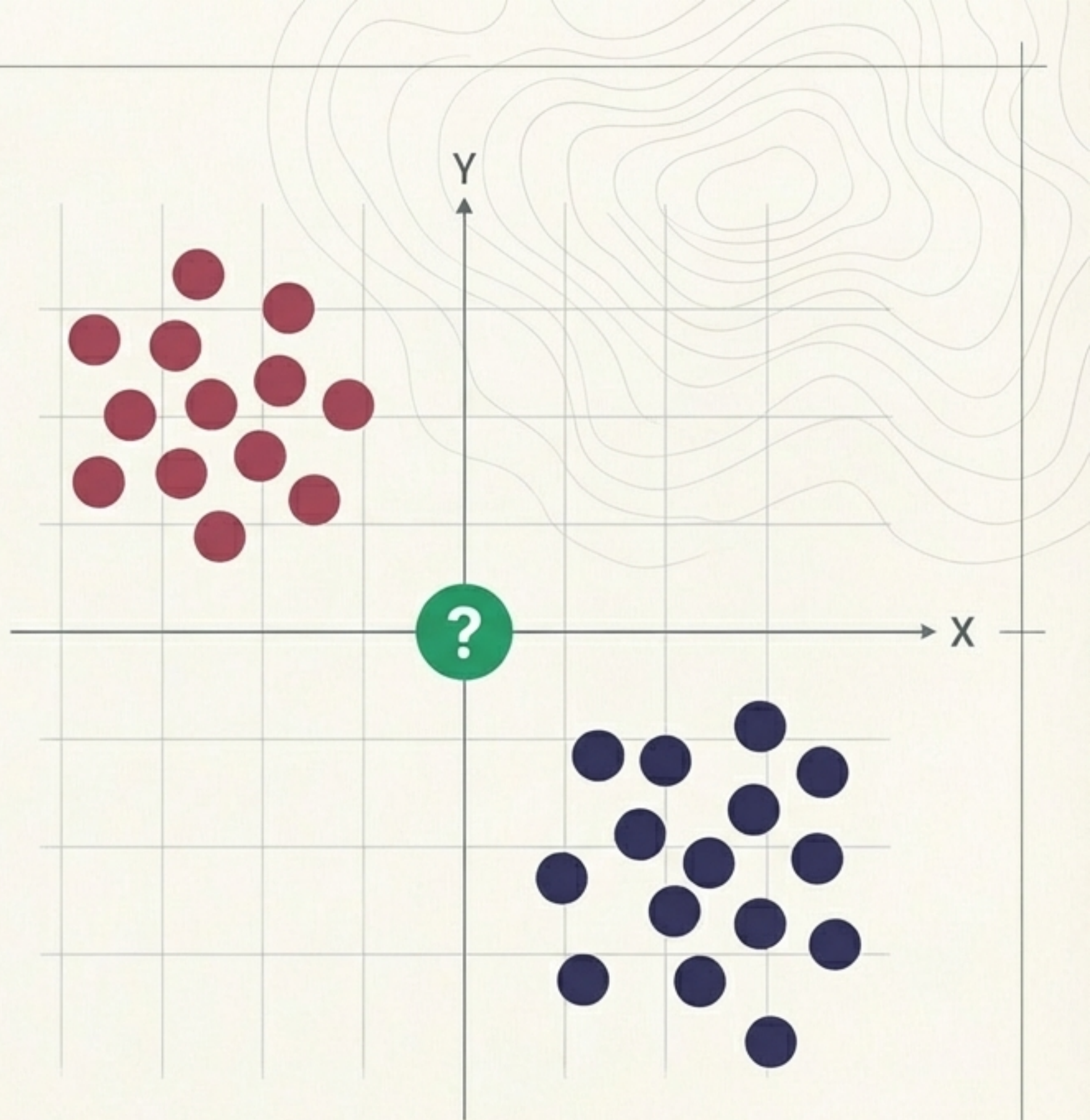
KNN은 수학적 계산보다 공간적 거리를 기반으로 한 ‘민주주의 투표’와 같습니다.
내 주변에 어떤 성향의 이웃이 가장 많은지 세어보고, 나의 소속을 결정합니다.

미지의 데이터가 등장하다

이미 분류된 파란색 그룹과
빨간색(마젠타) 그룹이 존재합니다.

이때, 중앙에 새로운 초록색 데이터가
나타났습니다.

이 인공지능은 초록색 데이터를
어느 그룹으로 분류해야 할까요?



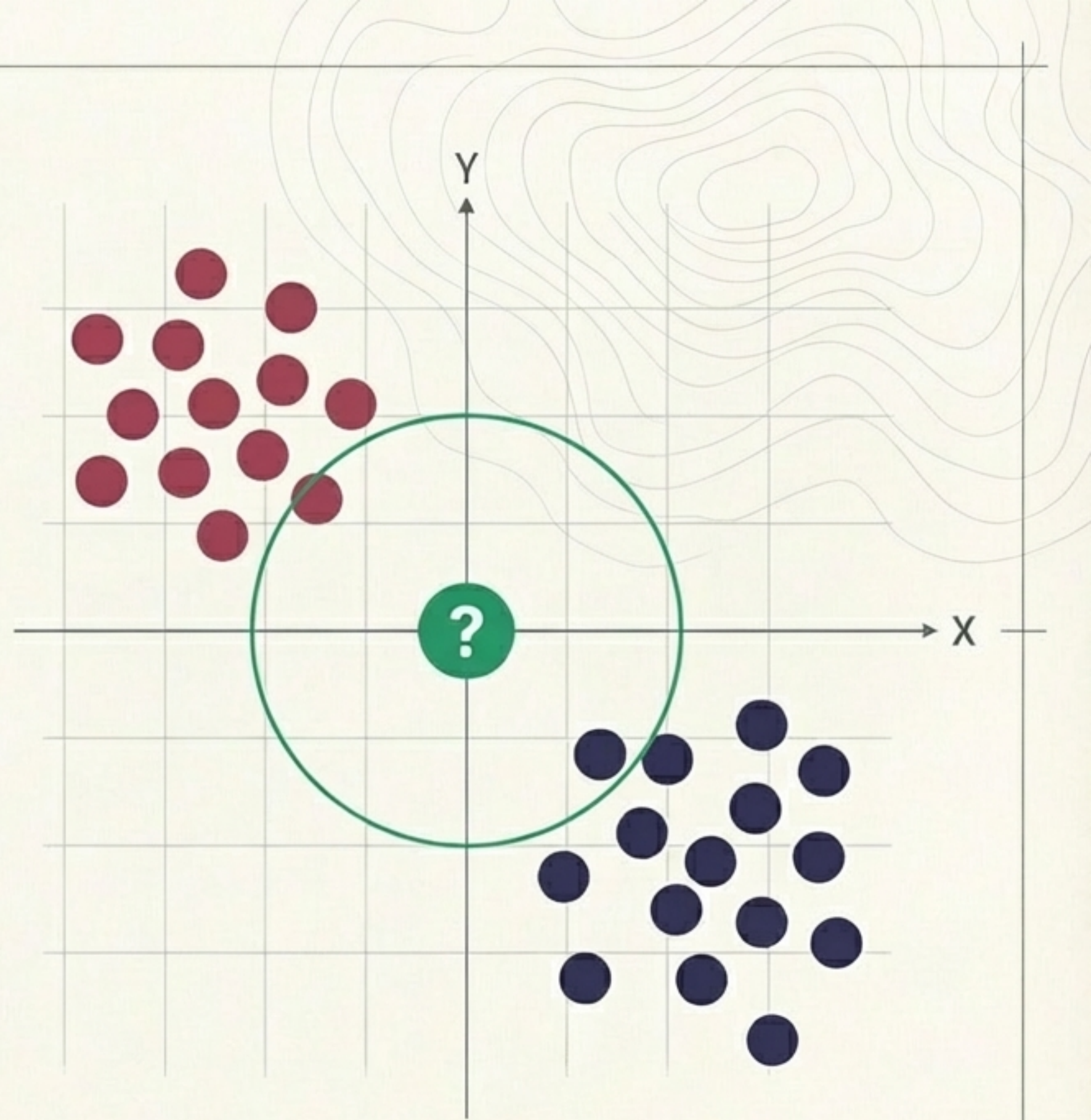
탐색 반경의 설정과 이웃 포착



K-Value Selector

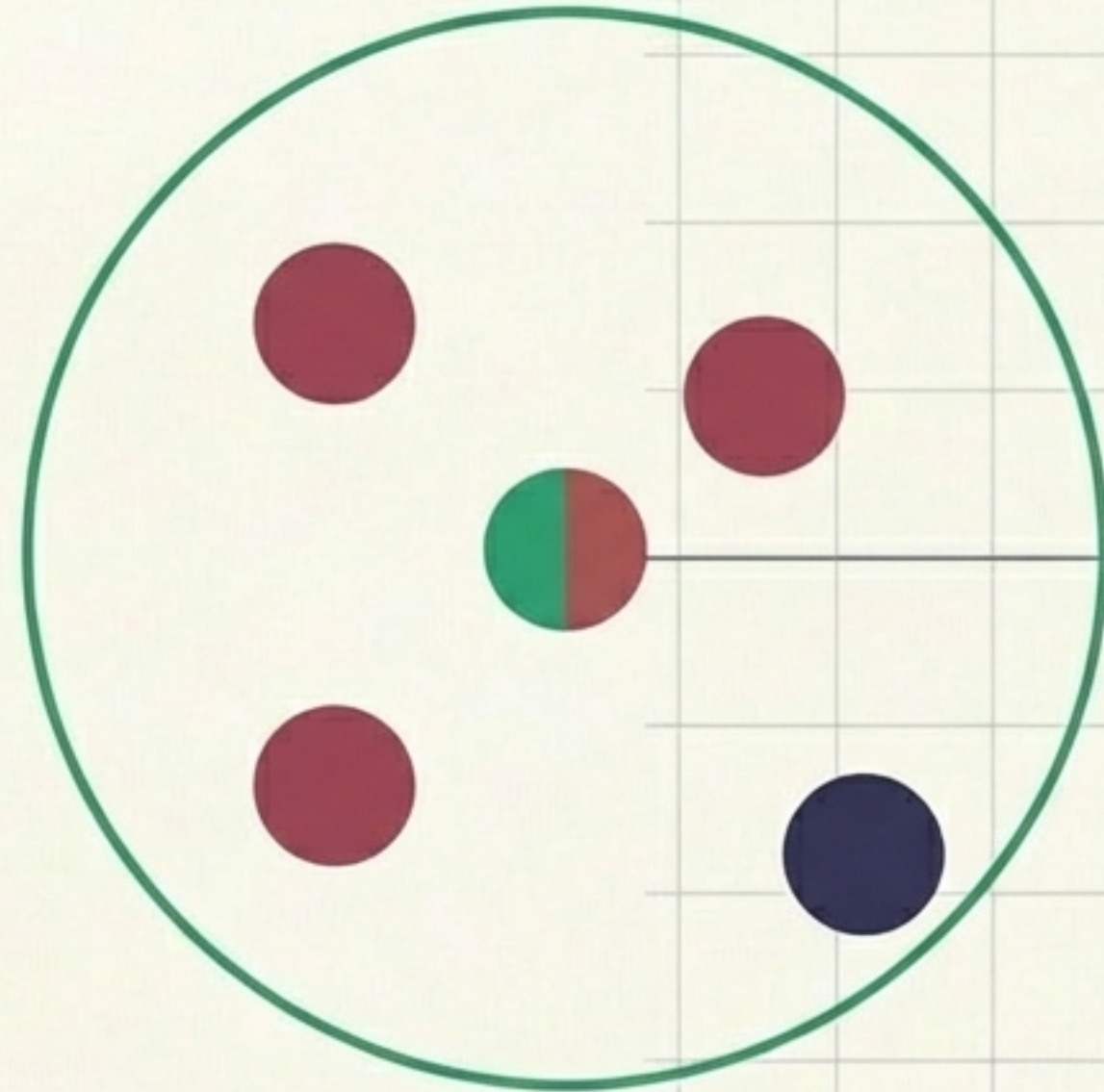
동점을 방지하기 위해
보통 3, 5, 7 같은
홀수를 사용합니다.

K값을 3으로 설정하면, 미지의
데이터로부터 가장 가까운 3개의 이웃만
탐색 반경에 포함시킵니다.



다수결 투표를 통한 최종 분류

포착된 3개의 이웃 중
마젠타 그룹이 더 많습니다.
따라서 새로운 데이터는
마젠타 그룹으로 최종
분류됩니다.



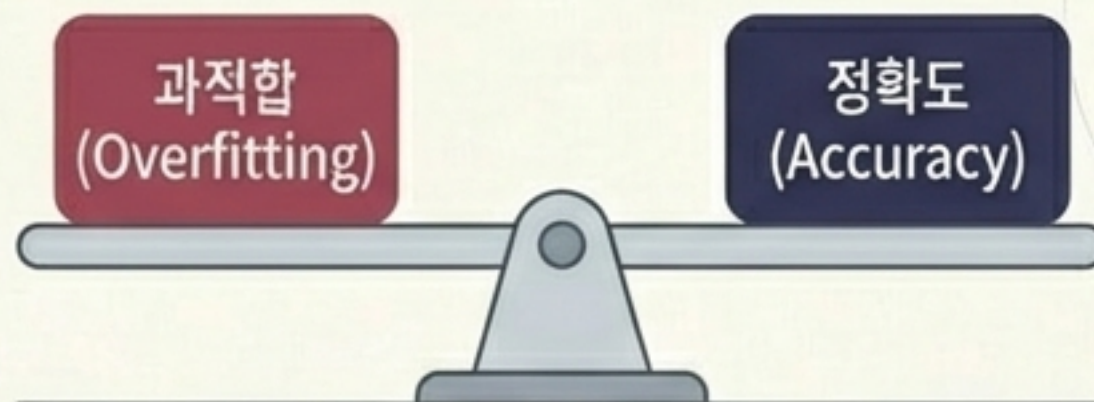
마젠타 그룹 (Magenta): 2 표

66%

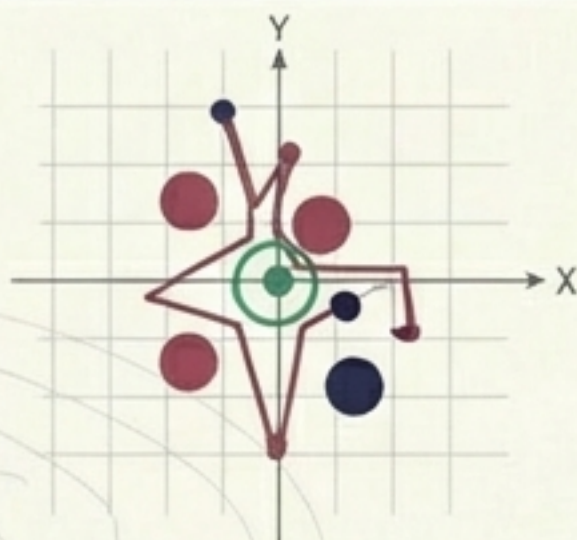
파란색 그룹 (Blue): 1 표

33%

K값의 딜레마 시소



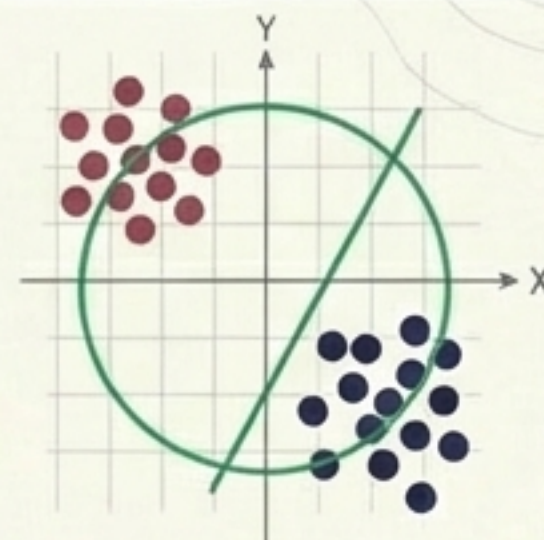
낮은 K값 (예: K=1)



과적합 가능성 (Overfitting): **높음**
일반화 정확도 (Accuracy): **낮음**

특징: 학습 데이터에는 완벽하지만, 노이즈나 예외값에 너무 민감하게 반응합니다.

높은 K값 (예: K=15)



과적합 가능성 (Overfitting): **낮음**
일반화 정확도 (Accuracy): **낮아질 수 있음**
특징: 너무 많은 데이터를 참고하여 경계가 흐려지고 정교한 분류가 어려워집니다.

K값의 딜레마 (과적합 시소): K값을 무조건 높이거나 낮추는 것이 정답은 아닙니다.
데이터의 특성에 맞는 최적의 균형점을 찾는 것이 핵심입니다.

이론에서 코드로: Scikit-learn 파이프라인

파이썬을 활용한 KNN 구현은 4단계의 직관적인 공정으로 이루어집니다.
복잡한 수학 대신 프레임워크의 도구를 조립하는 과정입니다.



Step 1 & 2: 데이터의 분리

두 개의 좌표(Input)와 결과값(Target)을 분리한 후, 모델을 가르칠 '학습 데이터'와 성능을 시험할 '테스트 데이터'로 나눕니다.

	Input Data (X) [좌표]	Target Data (Y) [그룹]
Train (학습용 80%)	Row 1: [2, 3]	Group A
	Row 2: [5, 1]	Group B
	Row 3: [7, 8]	Group A
	Row 4: [1, 9]	Group B
Test (평가용 20%)	Row 5: [4, 4]	Group A
	Row 6: [6, 2]	Group B

```
x, y = data_import()
```

```
train_x, test_x, train_y, test_y =  
    train_test_split(x, y)
```


Step 3: 핵심 엔진의 가동

KNeighborsClassifier 모듈을 불러옵니다. 여기서 가장 중요한 것은 `n_neighbors` 파라미터이며, 이것이 바로 우리가 앞서 배운 K값(탐색 반경)을 설정하는 다이얼입니다.



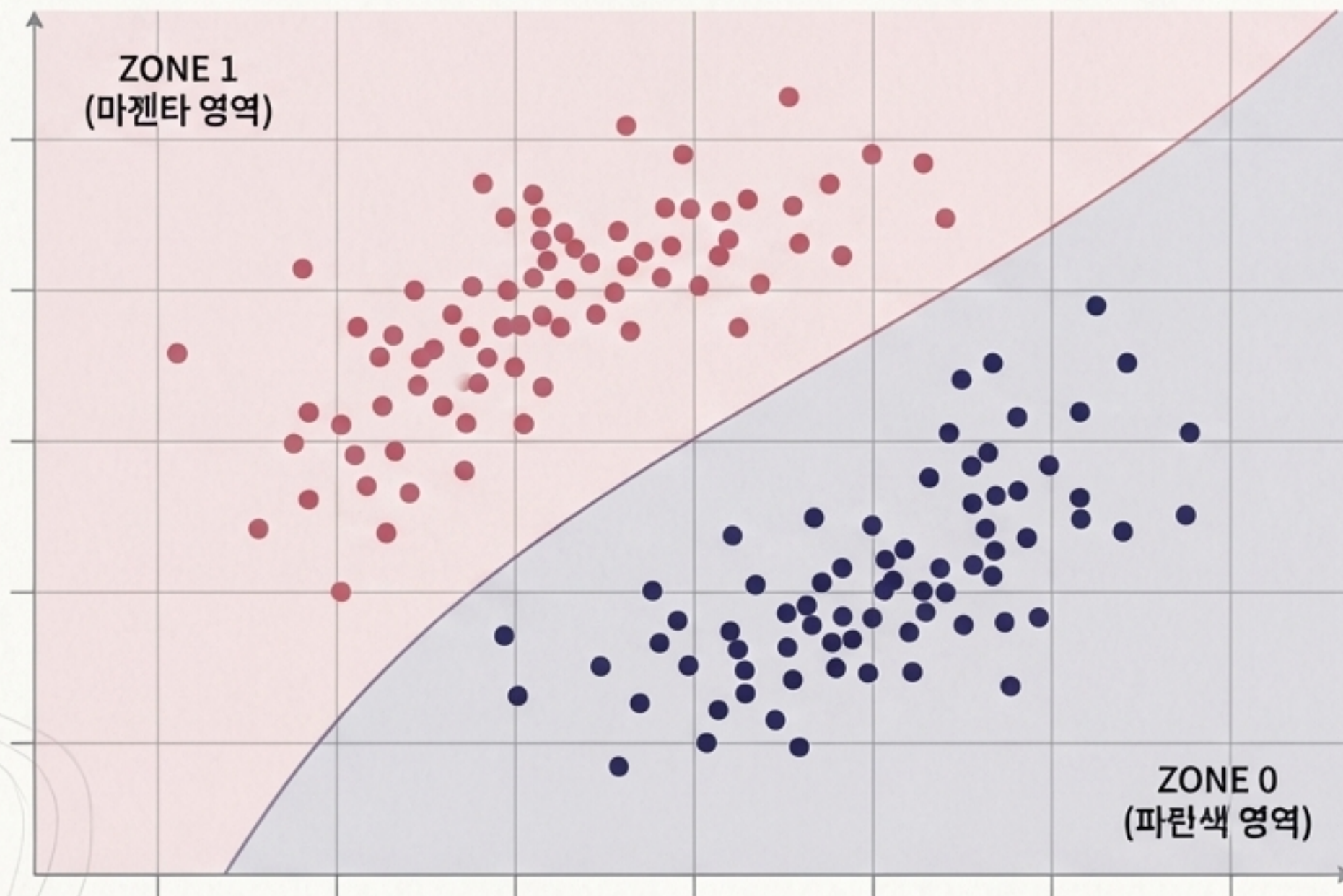
```
knn_model = KNeighborsClassifier(n_neighbors=3)
```

```
knn_model.fit(train_x, train_y)
```



학습이 완료된 데이터 공간

모델 학습이 끝나면, 컴퓨터는 2차원 공간을 두 개의 영역(0 그룹과 1 그룹)으로 명확히 분할하여 인식여 인식하게 됩니다.



이론과 실전의 증명 (Theory Meets Reality)

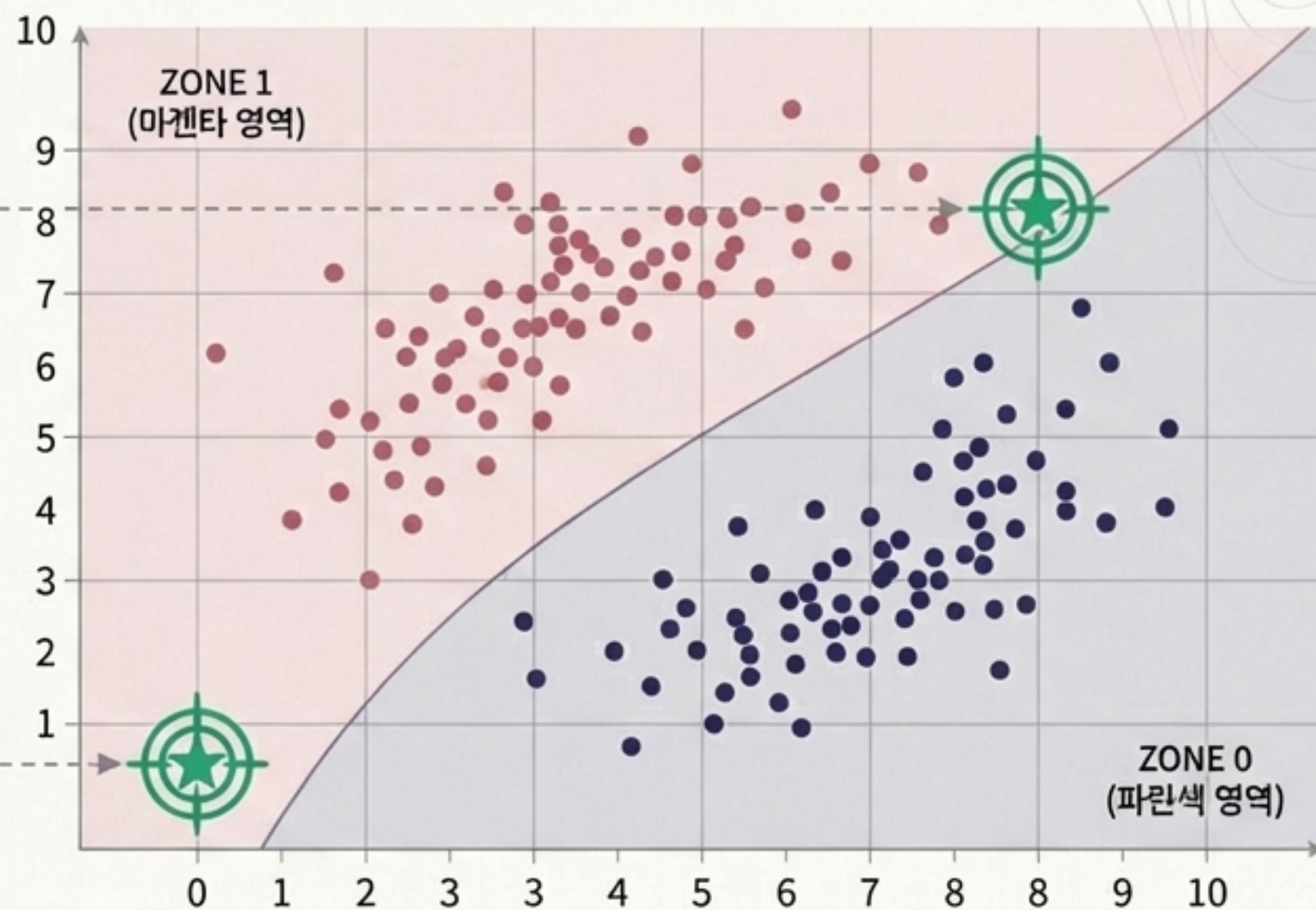
새로운 좌표 [8,8]과 [1,1]을 입력해 봅니다. 지도상의 위치를 확인하면, 주변 이웃의 다수결에 따라 [8,8]은 1그룹에, [1,1]은 0그룹에 정확히 속하게 됨을 시각적으로 증명할 수 있습니다.

CODE COMMANDS

```
predict([8,8]) → Output: 1
```

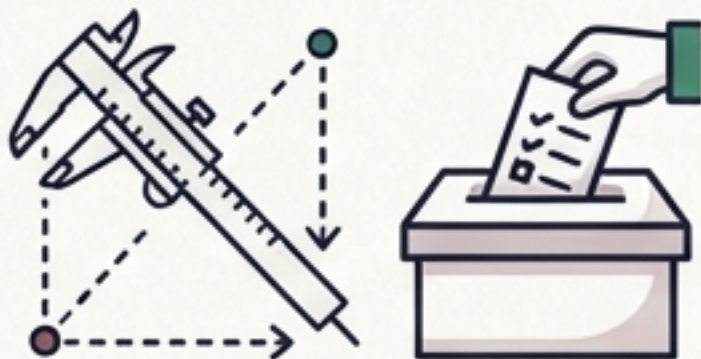
```
predict([1,1]) → Output: 0
```

THE MAP



KNN 한 장 요약 (Cheatsheet)

핵심 원리 (Core Concept)



거리 기반의 다수결 투표 알고리즘.
수학적 계산 대신 공간상에서 가장
가까운 이웃 K개의 그룹 비율로
소속을 결정합니다.

K값의 딜레마 (The Dilemma)



K값이 작으면:
노이즈에 민감하여
과적합(Overfitting)
발생 가능성 높음.

K값이 크면:
경계가 모호해져
일반화
정확도(Accuracy)
하락 위험.

핵심 파이썬 코드 (Core Code)



```
knn = KNeighborsClassifier  
(n_neighbors=k)  
knn.fit(X_train, y_train)  
knn.predict(X_new)
```

Scikit-learn을 통해
4단계(준비->분할->학습->예측)로
즉시 구현 가능.